

Úvod

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, 2021

Základy strukturovaného programování, ZSP
09/2021, Lekce 1

<https://moodle.vspj.cz/course/view.php?id=201662>



Co to je problém?

- Situace, která je nežádoucí a vyžaduje nějaké řešení.
- Řešení problémů (nebo také hledání řešení) patří mezi kognitivní schopnosti a dovednosti, schopnost řešit problémy je důležitou složkou inteligence každého člověka.
- Obvyklý postup při řešení problému:
 - identifikace problému - zjištění, že situaci nelze snadno zvládnout obvyklými způsoby
 - definice problému - vymezení problému
 - výběr strategie řešení - metoda pokusů a omylů, použití již existujícího řešení, strategie vhledu (porozumění problému), vlastní nové řešení
 - kontrola správnosti a efektivity řešení.
- Výpočetní problém – předpokládá řešení ve formě výpočtu.

Výpočetní problém

- Specifikace výpočetního problému má dvě části:
 - definici vstupu (vstupních dat) a
 - definici výstupu (výstupních dat).
- Instance výpočetního problému → nějaká konkrétní přípustná vstupní data. Obecně může k jedné instanci problému existovat několik správných výstupů (obecně se jedná o výpočet, představující relaci, ne vždy funkci).
- Příklad: problém řazení čísel (Sorting Problem for Numbers).
 - Vstup: Posloupnost n čísel $Inp = \langle a_1, \dots, a_n \rangle$.
 - Výstup: Taková permutace $Out = \langle a'_1, \dots, a'_n \rangle$ stejných čísel z Inp , pro kterou platí, že $a'_1 \leq \dots \leq a'_n$.
- Instance problému řazení je např. zadání: seřadte vzestupně posloupnost čísel $Inp = \langle 75, 11, 34, 176, 59, 6, 54 \rangle$. Správným výstupem řazení je pak posloupnost $Out = \langle 6, 11, 34, 54, 59, 75, 176 \rangle$.
- Pozn.: Zde je výstup určen jednoznačně, ale obecně to neplatí.

Algoritmizace

Algoritmus:

- Návod na provedení činnosti.
- Popis kroků (transformací) nutných pro převedení vstupů na výstupy.

Vlastnosti:

- Elementární – složen z elementárních kroků (realizovatelných úkonů).
- Deterministický (jednoznačný) – v každém kroku lze jednoznačně určit v jakém stavu se nacházíme (zda proces skončil/případně jaký další krok se vykoná).
- Konečný – počet kroků je konečný.
- Rezultativní – vede vždy ke správnému výsledku.
- Hromadný – předpis řešení lze použít na libovolnou úlohu dané skupiny.

Algoritmus vykonává (je nutné znát seznam elementárních kroků):

- Realizátor – Procesor.
- Umí provádět elementární kroky, umí určit, který krok následuje.

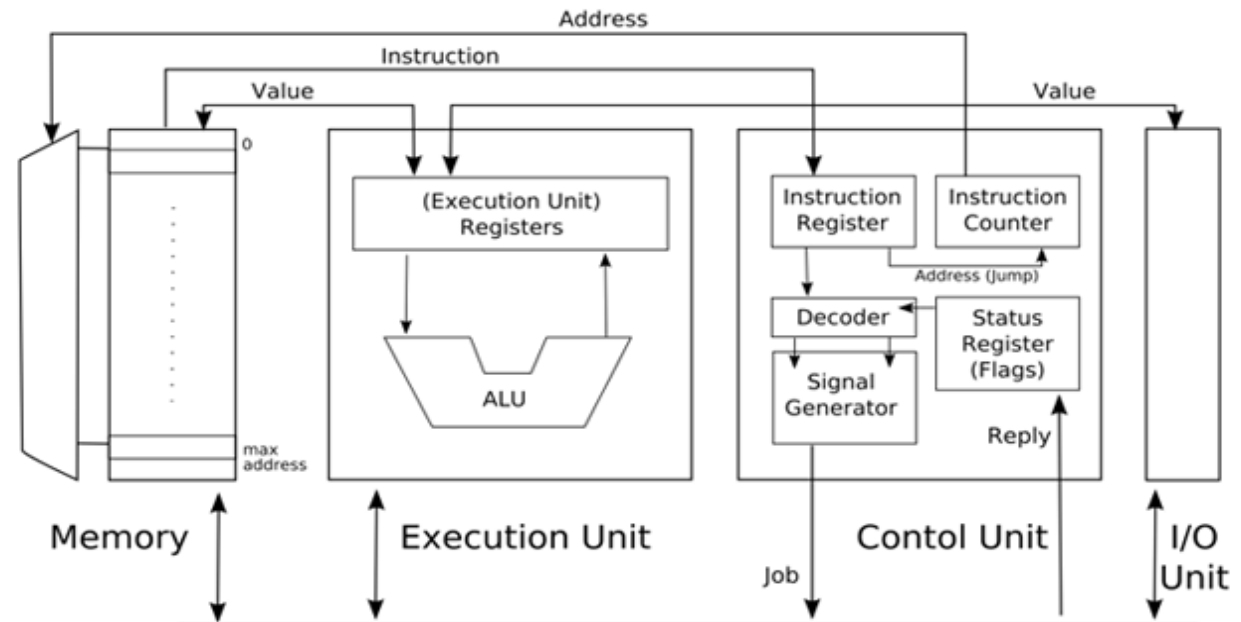
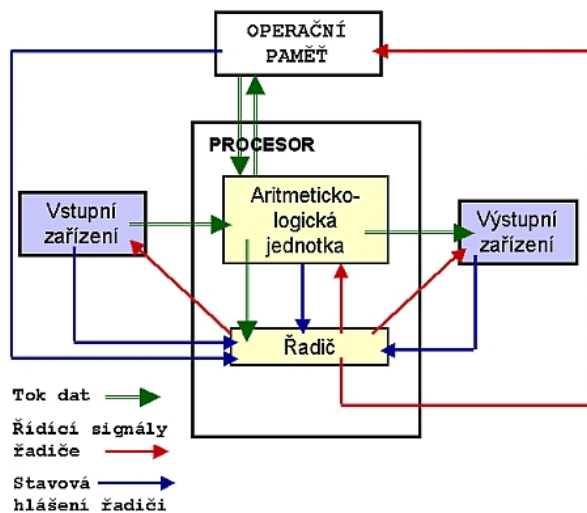
Výpočetní stroj

Von Neumannova architektura (1946):

- primitivní paměť (obsahuje data i program)
- sekvenční řízení výpočtu (čítač adres)



John von Neumann



Algoritmus

- Neformálně: algoritmus je jakýkoli dostatečně přesný popis postupu výpočtu – dostatečně přesný znamená:
 - v každém okamžiku postupu víme, který krok bude následovat (postup je deterministický),
 - pro libovolná vstupní data postup vždy skončí (je konečný) a
 - lze jej spustit pro všechny instance problému (je hromadný).
- Algoritmus řešící nějaký výpočetní problém je správný (korektní), pokud pro každou vstupní instanci skončí se správným výstupem. Správnost algoritmu se musí dokázat (proč?).
- Algoritmus není program.
 - Program může být realizací algoritmu v určitém výpočetním prostředí.
- Mezi algoritmy a výpočetními problémy nemusí být vztah jedna ku jedné – stejný problém lze zpravidla řešit různými algoritmy.
- Různé algoritmy pro stejný problém mohou mít různou složitost.

Korektnost algoritmu

- Korektní algoritmus A pro problém V :
 - pro každou instanci problému V , algoritmus A skončí v konečném čase se správným výstupem.
- Pak říkáme, že algoritmus A řeší problém V (nebo je řešením pro V).
- Čili: algoritmus A není korektní, pokud:
 - pro některou instanci problému neskončí (např. se “zacyklí”) nebo
 - pro některou instanci problému skončí s nesprávným výstupem.
- Můžeme pak nejvýše hovořit o jisté proceduře (postupu).
- Zde uvažujeme výhradně pouze korektní algoritmy – tj. algoritmy, které skutečně řeší odpovídající problém.
- Důkaz korektnosti nemusí být jednoduchý – instancí problému bývá zpravidla nekonečně mnoho – ověřit postup pro všechna data nelze. Je nutno použít sofistikovanější metody, např. matematickou indukci.

Číselné soustavy

- Počítač (program) pracuje s čísly – „de facto“ pracuje jen s čísly.
- Čísla vyjadřujeme v různých číselných soustavách – nejčastěji v desítkové (dekadické), ale pro počítač se tato soustava nehodí – paměť počítače není konstruována na ukládání dekadických cifer, ale skládá se z binárních buněk (bitů – binary digits), kam se dají uložit čísla 0 a 1.
- Proto je nutno dekadická čísla převádět do dvojkové soustavy, případně soustavy, která zahrnuje více bitů (oktalové, hexadecimální).
- Pro převod reprezentace čísel mezi různými pozičními soustavami můžeme definovat obecný algoritmus.
- Vyjádření čísla může usnadnit jeho zpracování, uvažte např. následující výpočet:
$$\text{CLXV} * \text{XXVIII} = ?$$
- Na druhé straně umíme spočítat, kolik je:
$$165 * 28 = 4620 \text{ (tj. MMMMDCXX)}$$

Algoritmus převodu mezi číselnými soustavami

Algoritmus převodu ze soustavy o základu A do soustavy o základu B:

- **Vstup:** buď X_A číslo vyjádřené v soustavě o základu A
- **Výstup:** hledáme vyjádření Y_B v soustavě o základu B
- **Postup:**
 1. Nastavíme $Y_B = ""$
 2. $Q = X_A/B$, $R = X_A \% B$ (postupně dělíme základem soustavy, do které číslo převádíme)
 3. $X_A = Q$, $Y_B = R \bullet Y_B$ (připojíme R do čela Y_B)
 4. Pokud $X_A = 0$, jdi na krok 5, jinak pokračuj krokem 2
 5. Výsledek je Y_B
- **Příklad:** Pokud převádíme dekadické číslo 123_{10} do dvojkové nebo hexadecimální soustavy, bude podle tohoto algoritmu mít vyjádření:
$$123_{10} = 1 \cdot 64 + 1 \cdot 32 + 1 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 1111011_2$$
$$123_{10} = 7 \cdot 16 + 1 \cdot 11 = 7B_{16}$$
- Pozn.: Jedná se o korektní algoritmus, neboť vždy skončí se správným výsledkem.

Programy a programovací jazyky

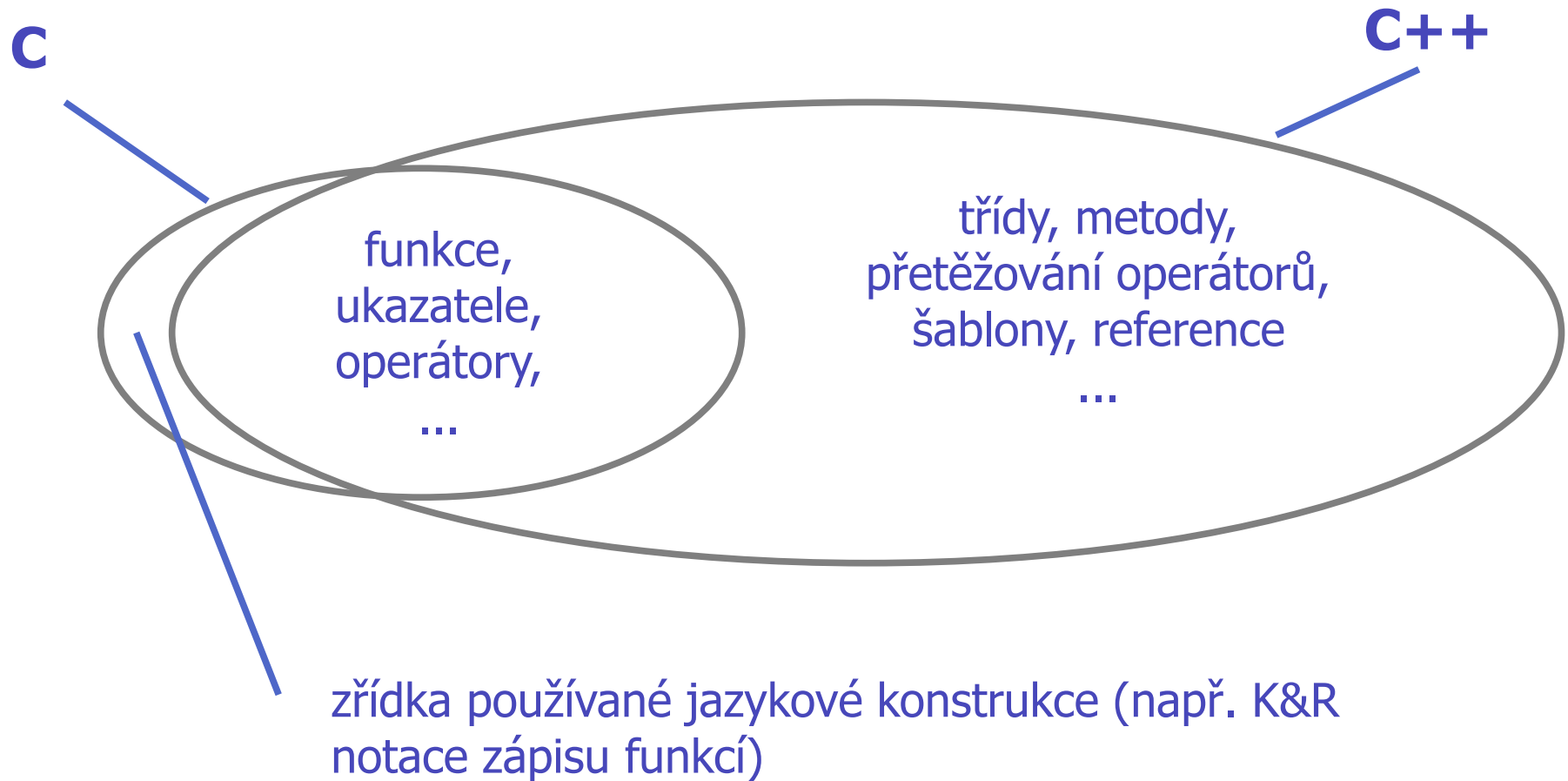
Program je předpis pro provedení určitých akcí počítačem zapsaný v programovacím jazyku (je to vždy zápis algoritmu?).

- Programovací jazyky:
 - strojově orientované (strojový jazyk = jazyk fyzického procesoru),
 - assembler (jazyk symbolických adres),
 - vyšší jazyky:
 - **imperativní** (příkazové, procedurální),
 - **neimperativní** (např. funkcionální, logické).
- Hlavní rysy imperativních jazyků (např. C, C++, Java, Pascal, Basic, ...):
 - zpracovávané údaje mají formu **datových objektů** různých typů, které jsou v programu reprezentovány pomocí **proměnných** resp. **konstant**, uložených v **paměti** počítače,
 - **program** obsahuje deklarace a příkazy,
 - **deklarace** definují význam jmen (identifikátorů),
 - **příkazy** předepisují akce s datovými objekty a způsob řízení výpočtu.

Historie C a C++

- Jazyk C původně vznikl jako systémový jazyk pro operační systém Unix (aby se nemusel psát v assembleru).
- Vycházel z jazyka BCPL (Martin Richards, Cambridge).
- Původní verze: jazyk B (Ken Thompson, PDP-7, 1970).
- Při přechodu na PDP-11: jazyk C (Dennis Ritchie a spol. - 1972)
- 1978 - Kernighan, Ritchie: The C Programming Language (K&R C)
- 1982 - ANSI C - X3J11 (ISO WG14 1990, doplňky ISO/IEC 9899, 1999 tzv. ISO C99, 2011 - C11).
- Později vznikají různá objektová rozšíření C (Objective C, C++, ..).
- C++ navrhl Bjarne Stroustrup, 1983, AT&T Bell Laboratories.
- 1985 Bjarne Stroustrup: The C++ Programming Language.
- 1990 Bjarne Stroustrup: The Annotated C++ Reference Manual.
- 1998 - ISO WG21, ISO/IEC 14882 (ANSI C++ - X3J16, 1998).
- 2011 - Rozšíření C11, C++11, dále 2014 – C++14, 2017 – C++17, atd. C++20

Vztah C a C++



Základní charakteristika jazyka C

- Procedurální vyšší programovací jazyk.
- Některé rysy nižších jazyků (assembleru – zejména C).
- Jednoduché (skalární) datové typy: celočíselné, racionální, ukazatelé.
- Strukturované datové typy: pole, řetězce, struktury.
- Bohaté operační struktury: aritmetické, logické, bitové, přiřazení.
- Základní řídicí struktury: podmíněné příkazy, přepínače, cykly, bloky, funkce.
- Vstup/výstup a ovládání dynamické volné paměti není součástí jazyka, ale standardních knihoven.
- Stručná (někdy nepřehledná) syntaxe.
- Program se skládá z modulů nezávisle překládaných, sestavených do řešení.
- Modul = zdrojový soubor obsahující deklarace a definice.
- Vstupním bodem programu je funkce **main**.

Program v C je sada funkcí

`main`

`funkce1`

`funkce2`

`funkceN`

Jedna z nich se jmenuje **main** (nebo obdobně)

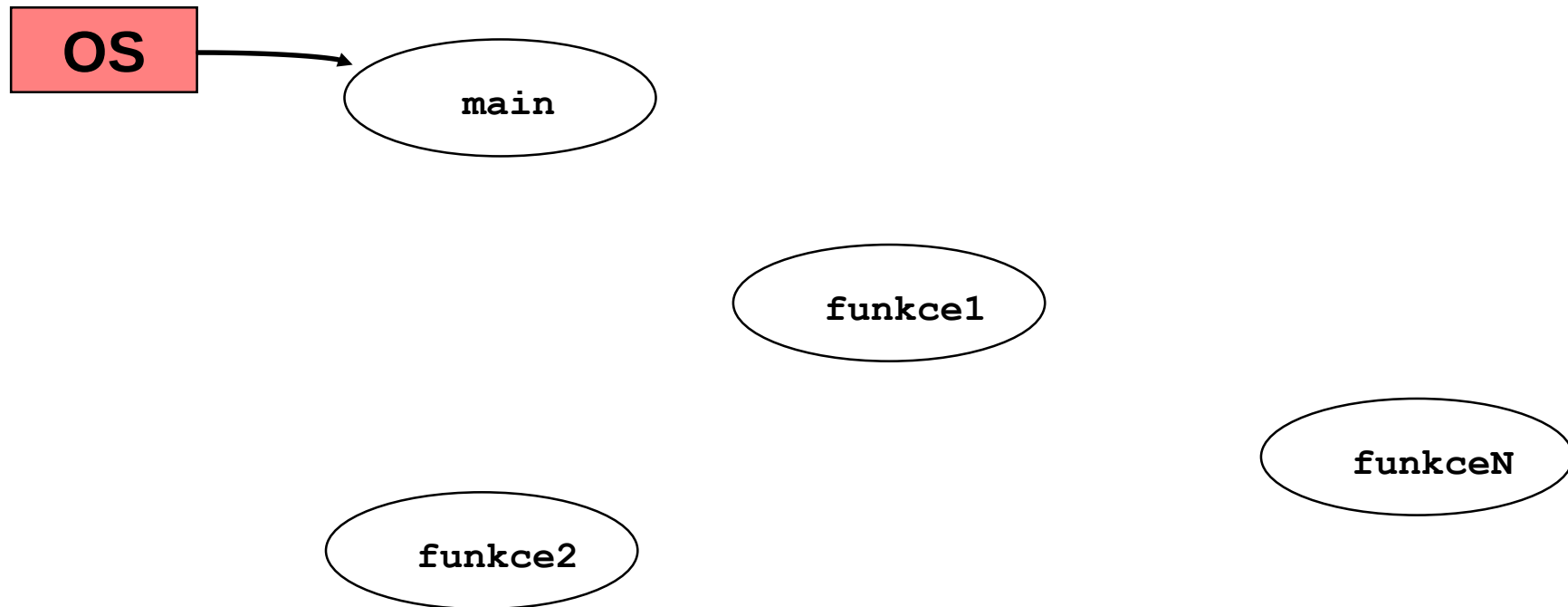
main

funkce1

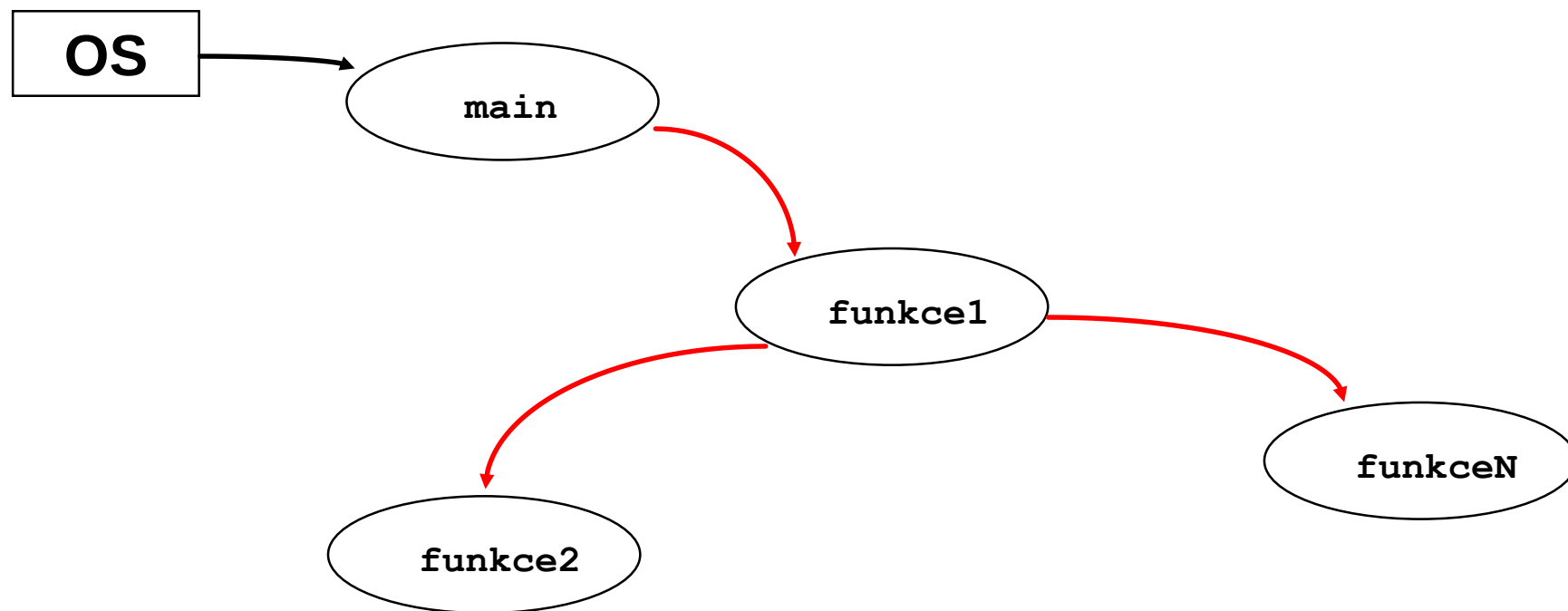
funkce2

funkceN

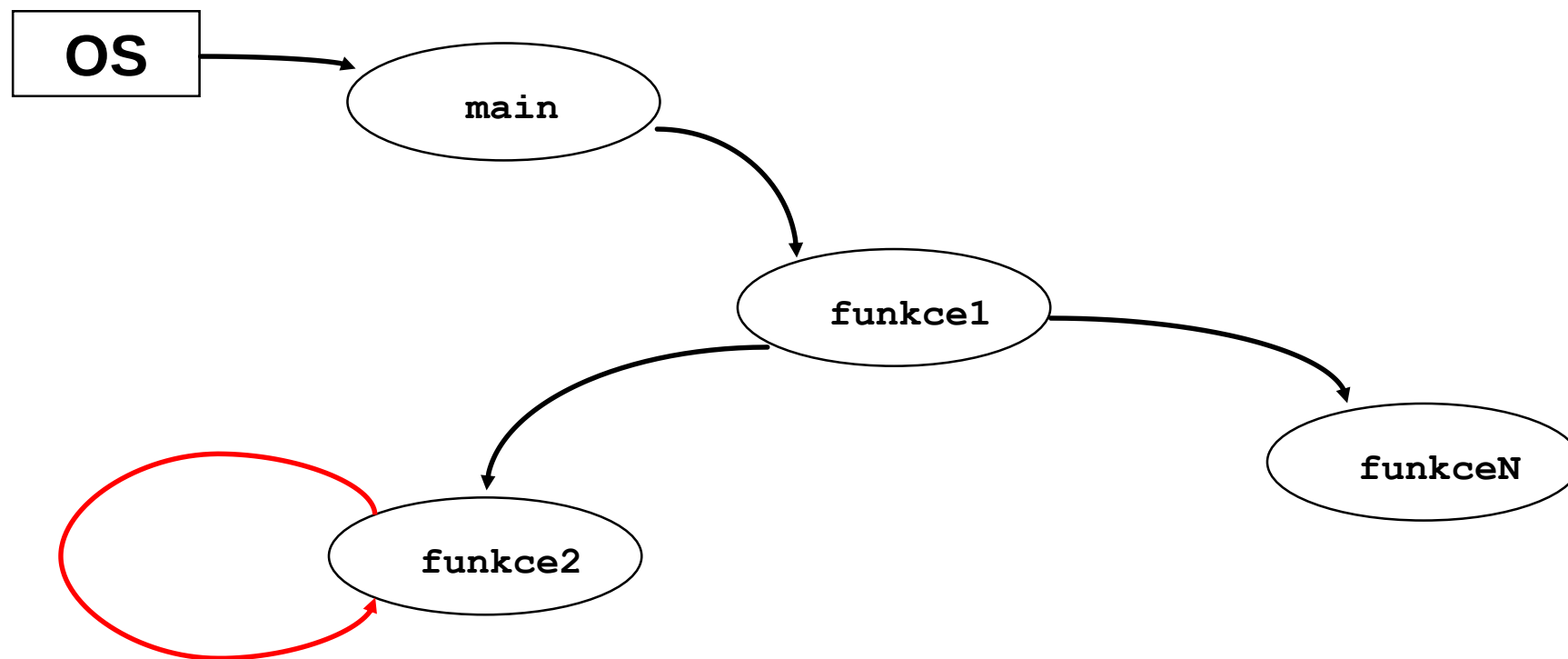
Funkci `main` spustí OS (shell)



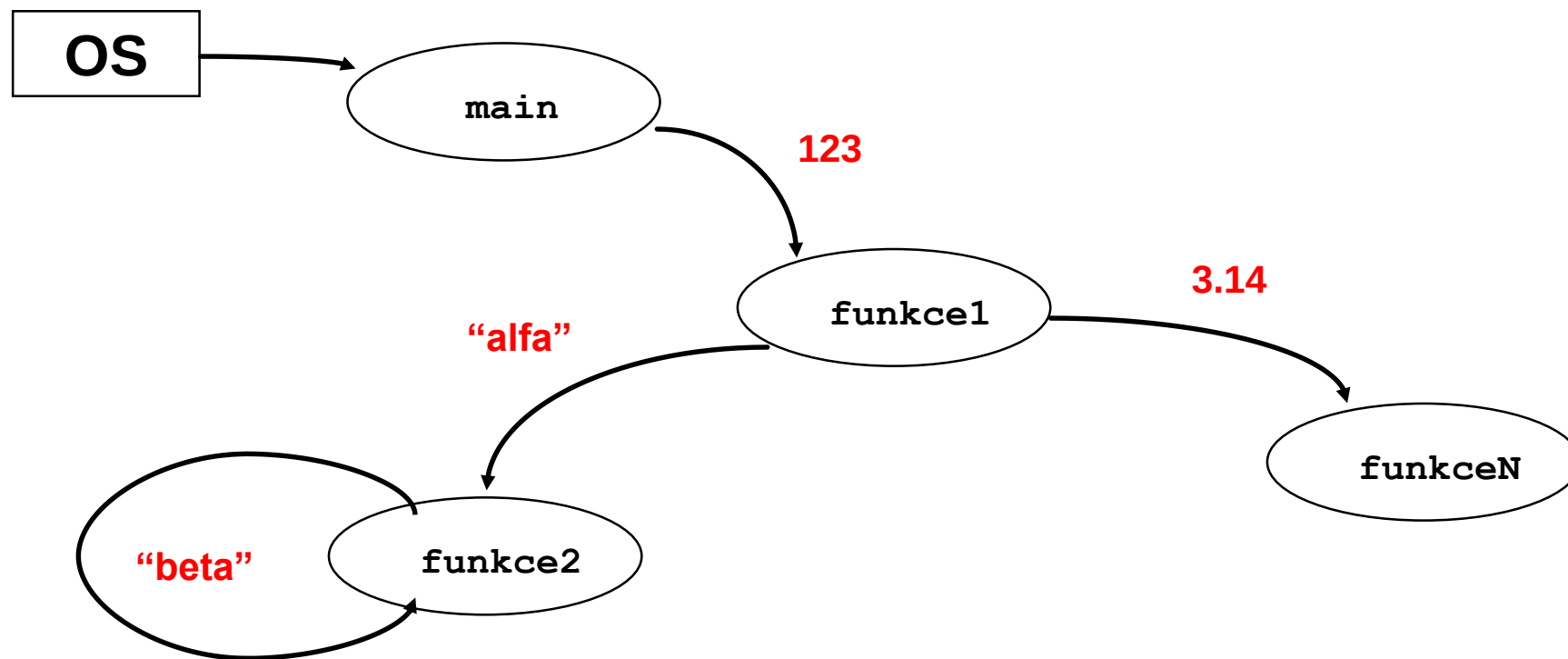
Funkce spolu komunikují (volají se)



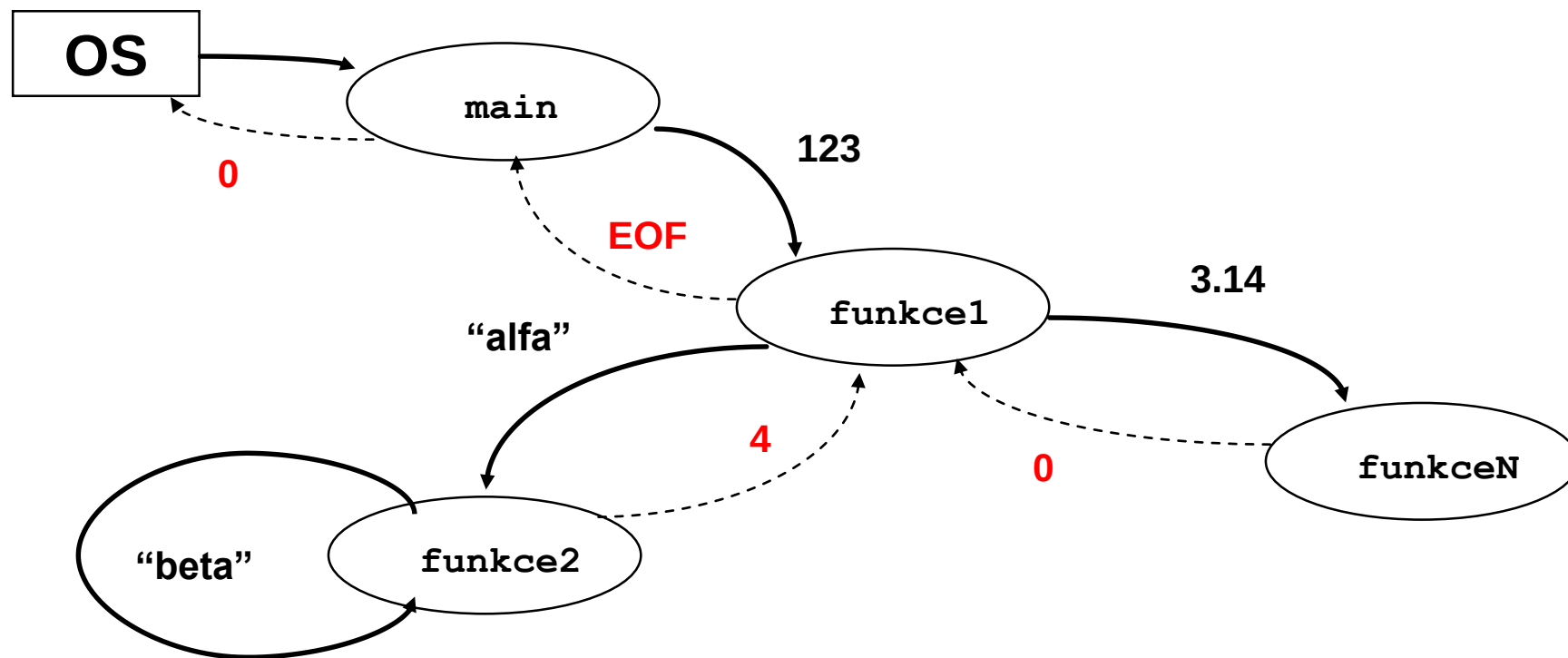
Případně i rekurzivně



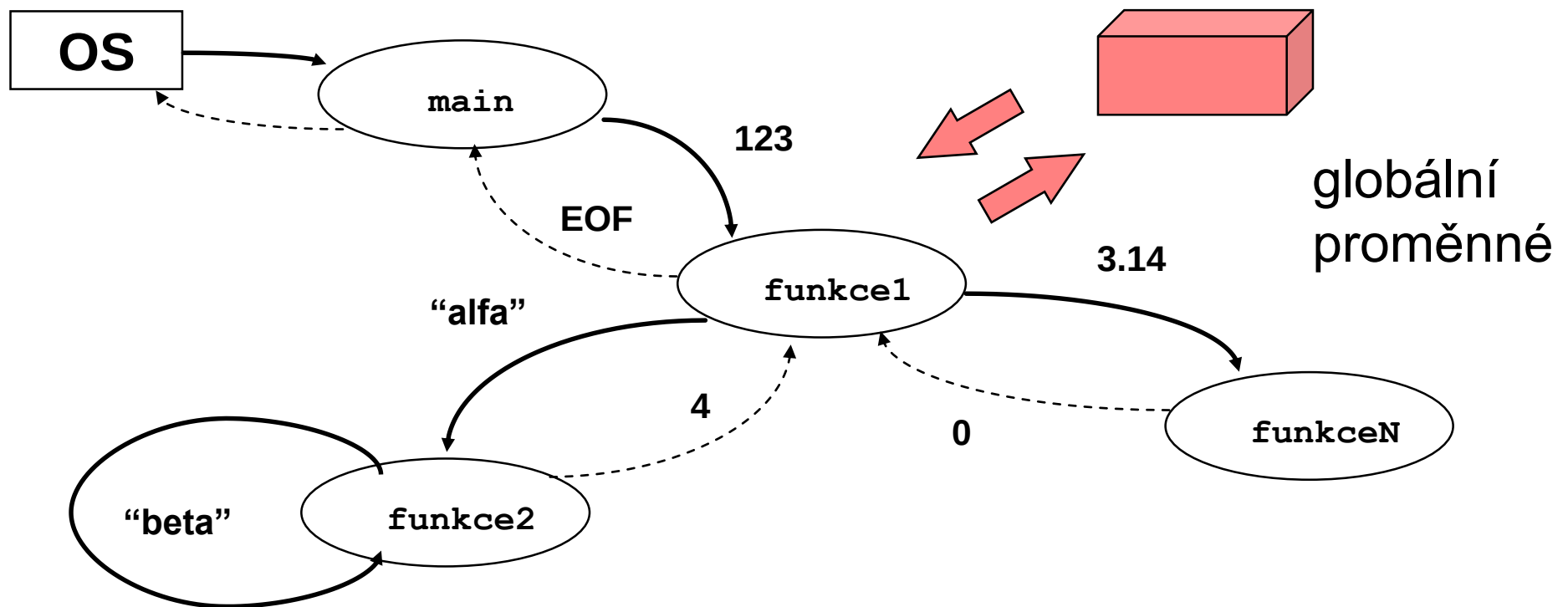
Předávají si parametry



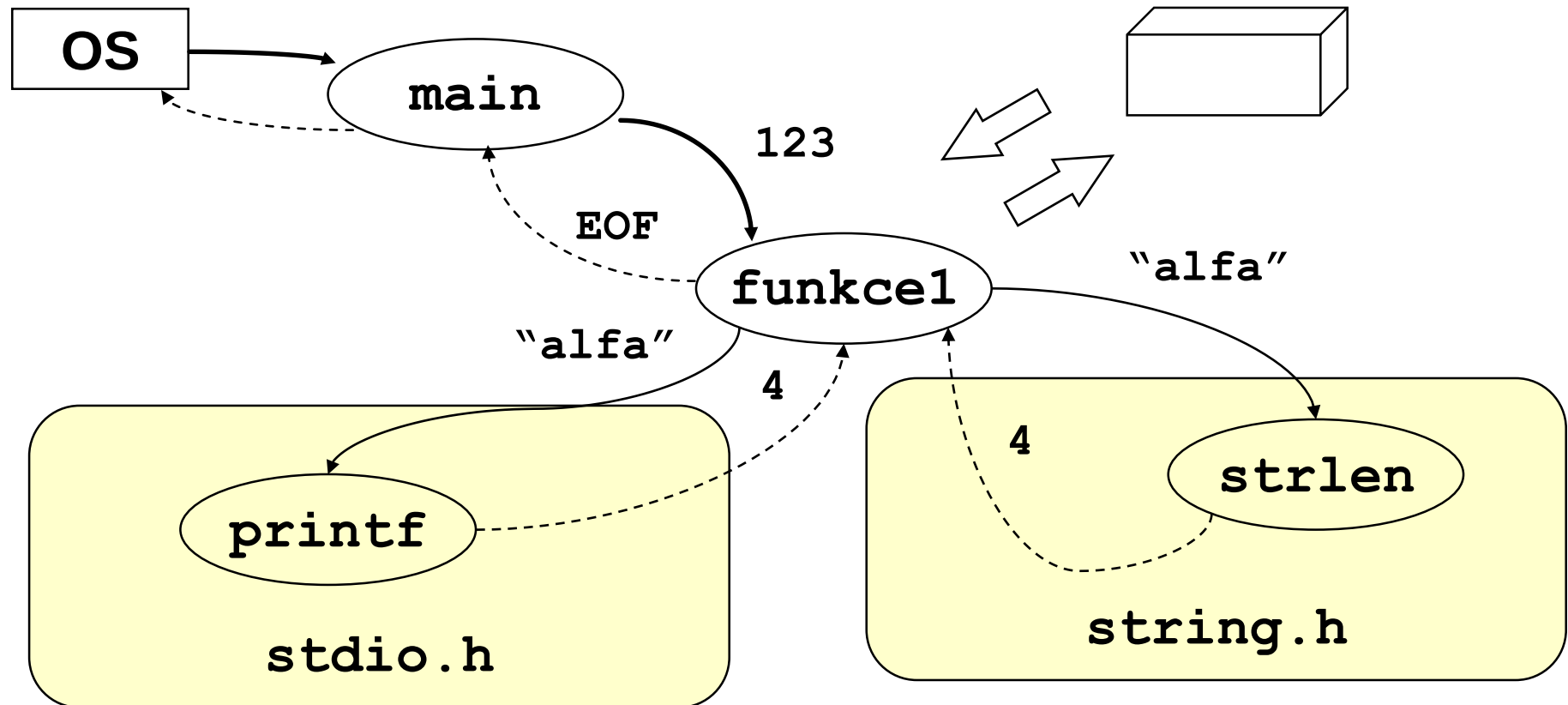
A vracejí výsledky



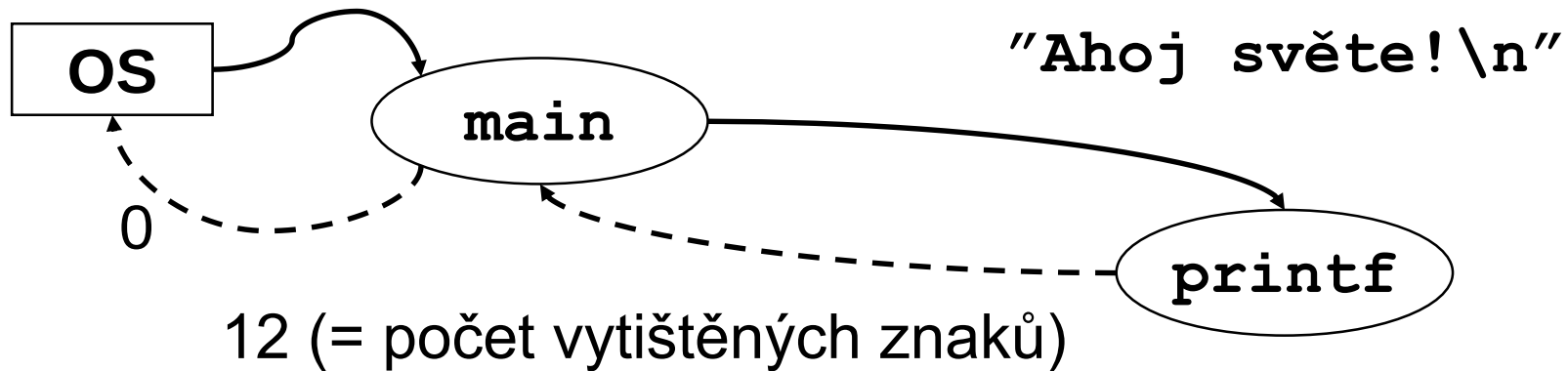
Navíc existuje globální paměť



Některé funkce jsou připraveny v knihovnách funkcí



Příklad “Ahoj světe!” (v C)



```
#include <stdio.h>
int main()
{
    printf("Ahoj světe!\n");
    return 0;
}
```

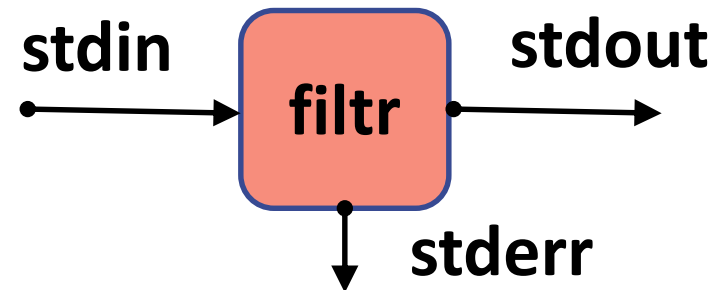
Vstup a výstup v C

- Každý program se chápe jako filtr – filtruje vstupní data na výstupní.
- Každý program proto má standardní vstup (stdin – standard input), standardní výstup (stdout – standard output) a výstup pro hlášení chyb (stderr – standard errors).
- Vstup pomocí scanf

```
scanf("%d",&x);
```
- Výstup pomocí printf

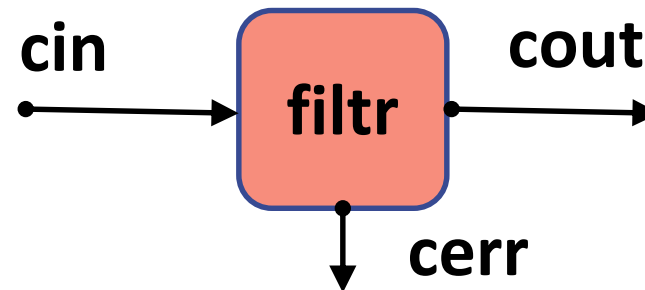
```
printf("%d",x);
```
- Konec řádku: `\n`

```
printf("\n");
```

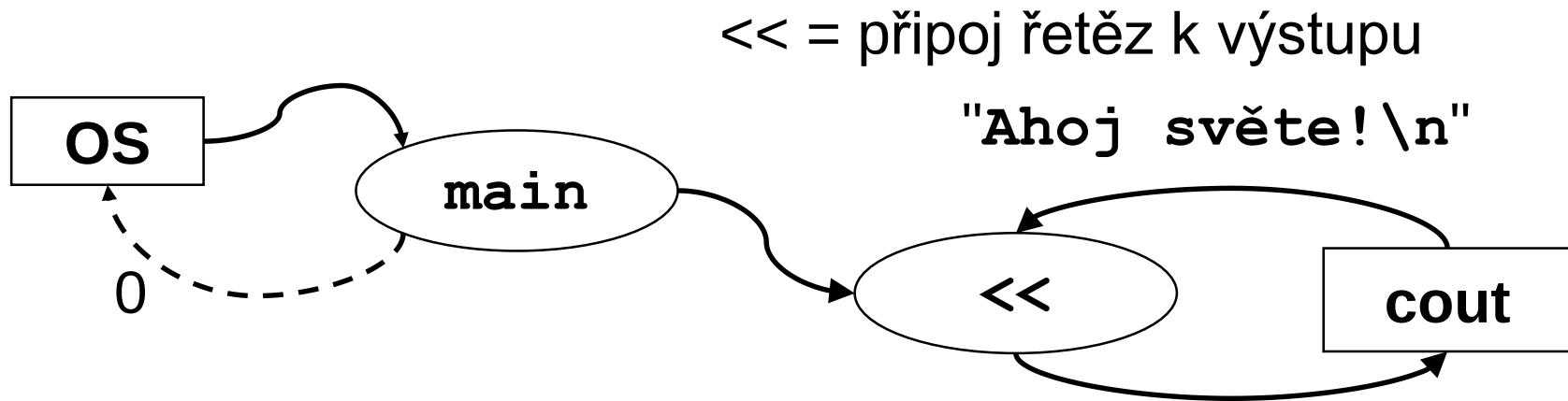


Vstup a výstup v C++

- Každý program se opět chápe jako filtr – filtruje vstupní data na výstupní.
- Každý program proto má standardní vstup (cin – console input), standardní výstup (cout – console output) a výstup pro hlášení chyb (cerr – console errors).
- Vstup pomocí >>
`cin >> x;`
- Výstup pomocí <<
`cout << x;`
- Konec řádku: endl
`cout << endl;`

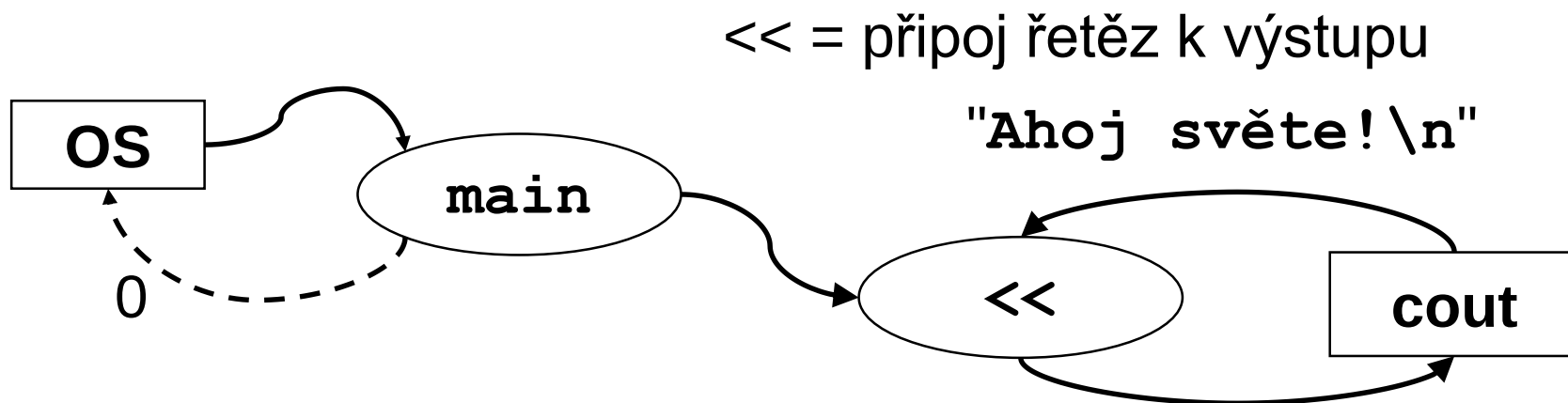


Příklad "Ahoj světe!" v C++



```
#include <iostream>
using namespace std;
int main()
{
    cout << "Ahoj světe! \n";
    return 0;
}
```

Příklad "Ahoj světe!" v C++ - alternativa



```
#include <iostream>

int main()
{
    std::cout << "Ahoj světe! \n";
    return 0;
}
```

Preprocesor

- Povinná součást prostředí C
- Provádí textové úpravy před vlastním překladem
- Řídí se příkazy pro preprocesor – **direktivami**
- Formát direktivy:
 #direktiva parametry
- Příklady direktiv:
 - Vložení textu
 #include <soubor>
 #include "soubor"
 - Definice makra
 #define jméno text
 #define jméno(parametry) text
 - Podmíněný překlad, parametry překladu (později)

Preprocesor (pokrač.)

Příklad: *Vložení souboru*

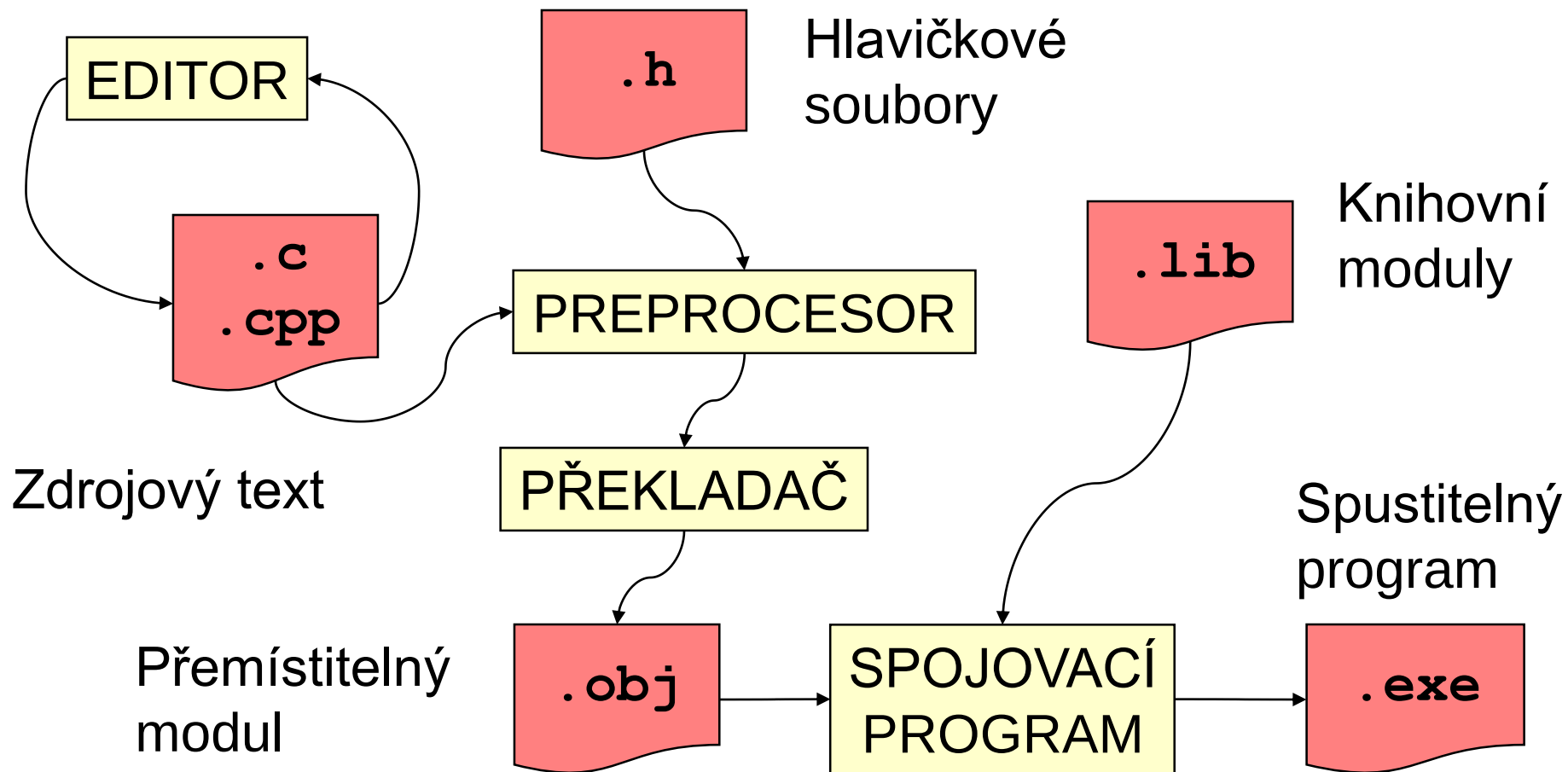
```
#include <stdio.h>
```

```
...
```

```
printf("Ahoj světe!\n");
```

funkce **printf** je součástí knihovny **stdio** (standardní vstup a výstup) a je popsána (deklarována) v hlavičce **stdio.h**

Fáze překladač



Ukázka programu v C

```
/* Ahoj světe */
```

komentář

```
#include <stdio.h>
```

použití
knihovny
stdio.h

```
int main()  
{  
    printf("Ahoj světe!\n");  
    return 0;  
}
```

funkce **main**
používá funkci
printf

Formát zápisu programu v C není předepsán

```
#include <stdio.h>
int main(int t, int _, char *a){return!0<t?t<3?main(-79,-13,a+main(-87,1-_,
main(-86,0,a+1)+a)):1,t<_?main(t+1,_,a):3,main(-94,-27+t,a)&&t==2?_<13?
main(2,_,+1,"%s %d %d\n"):9:16:t<0?t<-72?main(_,t,
"@n'+,#'/*{}w+/w#cdnr/+,{}r/*de}+,/*{*+,/w{%+,/w#q#n+,/#{l,+,/n{n+,/+#n+,/#\
;q#n+,/+k#;*+,/'r : 'd*'3,){w+K w'K:'+'e#';dq#'\
q#'+d'K#!/+k#;q#'}eKK#}w'r}eKK{nl}'/#;#q#n')})#}w')}){nl}'/+#n';d}rw' i;# \
){nl}!/n{n#'; r{#w'r nc{nl}'/#{l,+'K {rw' iK{;[{nl}'/w#q#n'wk nw' \
iwk{KK{nl}!/w{% 'l##w# ' i; :{nl}'/*{q#'ld;r'}{nlwb!/*de}'c \
;;{nl}'-{}rw}'/+,)##'*}#nc,',#nw}'/ +kd'+e}+;#rdq#w! nr/' ' ) }+}{rl#{n' ' )# \
}' +}##(!!/" )
:t<-50?_==*a?putchar(31[a]):main(-65,_,a+1):main((*a=='/')+t,_,a+1)
:0<t?main(2,2,"%s"): *a=='/' | | main(0,main(-61,*a,
"!ek;dc i@bK'(q)-[w]*%n+r3#l,{: \nuwloca-O;m .vpbks,fxntdCeghiry"),a+1);}
```

Zdroj: Wikipedia, International Obfuscated C Code Contest

Výstup tohoto programu

On the first day of Christmas my true love gave to me a partridge in a pear tree.

On the second day of Christmas my true love gave to me two turtle doves and a partridge in a pear tree.

On the third day of Christmas my true love gave to me three French hens, two turtle doves and a partridge in a pear tree.

On the fourth day of Christmas my true love gave to me four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

On the fifth day of Christmas my true love gave to me five gold rings; four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

On the sixth day of Christmas my true love gave to me six geese a-laying, five gold rings; four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

On the seventh day of Christmas my true love gave to me seven swans a-swimming, six geese a-laying, five gold rings; four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

On the eighth day of Christmas my true love gave to me eight maids a-milking, seven swans a-swimming, six geese a-laying, five gold rings; four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

On the ninth day of Christmas my true love gave to me nine ladies dancing, eight maids a-milking, seven swans a-swimming, six geese a-laying, five gold rings; four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

On the tenth day of Christmas my true love gave to me ten lords a-leaping, nine ladies dancing, eight maids a-milking, seven swans a-swimming, six geese a-laying, five gold rings; four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

On the eleventh day of Christmas my true love gave to me eleven pipers piping, ten lords a-leaping, nine ladies dancing, eight maids a-milking, seven swans a-swimming, six geese a-laying, five gold rings; four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

On the twelfth day of Christmas my true love gave to me twelve drummers drumming, eleven pipers piping, ten lords a-leaping, nine ladies dancing, eight maids a-milking, seven swans a-swimming, six geese a-laying, five gold rings; four calling birds, three French hens, two turtle doves and a partridge in a pear tree.

Výstup tohoto programu (česky)

Na první den Vánoc mi moje pravá láska dala koroptev v hrušce (Ježíš Kristus).

Na druhý den Vánoc mi moje pravá láska dala dvě hrdličky (Starý a Nový zákon) a koroptev v hrušce.

Na třetí den Vánoc mi moje pravá láska dala tři francouzské slepice (víra, naděje, láska), dvě hrdličky a koroptev v hrušce.

Na čtvrtý den Vánoc mi moje pravá láska dala čtyři volající ptáky (evangelisté), tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

Na pátý den Vánoc mi moje pravá láska dala pět zlatých prstenů (5 knih Mojžíšových), čtyři volající ptáky, tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

Šestý den Vánoc mi má pravá láska dala šest snášejších hus (šest dní stvoření světa), pět zlatých kroužků, čtyři volající ptáky, tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

Sedmý den Vánoc mi moje pravá láska dala sedm plovoucích labutí (sedm darů Ducha svatého), šest snášejších hus, pět zlatých prstenů, čtyři volající ptáky, tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

Osmého dne Vánoc mi moje pravá láska dala osm dojících služek (osm blahoslavenství), sedm plovoucích labutí, šest snášejších hus, pět zlatých prstenů, čtyři volající ptáky, tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

V devátý den Vánoc mi moje pravá láska dala devět tančících dam (ovoce Ducha svatého), osm dojících služek, sedm plovoucích labutí, šest snášejších hus, pět zlatých kroužků, čtyři volající ptáky, tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

Desátý den Vánoc mi má pravá láska dala deset skákajících pánů (Desatero přikázání), devět tančících dam, osm dojících služek, sedm plovoucích labutí, šest snášejších hus, pět zlatých kroužků, čtyři volající ptáky, tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

Jedenáctého dne Vánoc mi moje pravá láska dala jedenáct dudajících dudáků (jedenáct věrných apoštolů), deset skákajících pánů, devět tančících dam, osm dojících služek, sedm plovoucích labutí, šest snášejších hus, pět zlatých kroužků, čtyři volající ptáky, tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

Na dvanáctý den Vánoc mi moje pravá láska dala dvanáct bubnujících bubeníků (dvanáct bodů víry apoštolů), jedenáct dudajících dudáků, deset skákajících pánů, devět tančících dam, osm dojících služek, sedm plovoucích labutí, šest snášejších hus, pět zlaté prsteny, čtyři volající ptáky, tři francouzské slepice, dvě hrdličky a koroptev v hrušce.

Program

- Program P je zápis (implementace) algoritmu A v nějakém programovacím jazyce, spustitelný (proveditelný) na nějakém počítači s nějakým OS a SW prostředím.
- U návrhu a implementace programu nás zajímají SW inženýrské otázky, jako např.:
 - modularita,
 - ošetření výjimek,
 - datová abstrakce a ošetření vstupu a výstupu,
 - dokumentace.
- Jednomu algoritmu A mohou odpovídat různé programy $P_1, P_2, \dots$ dokonce v různých programovacích jazycích.
- Složitosti provedení jednotlivých programů P_i téhož algoritmu se mohou lišit v závislosti na architektuře počítače — nikoli ale řádově.

Tady jsme skončili

Výpočetní problém - příklad

- **Úloha:** najděte největšího společného dělitele dvou přirozených čísel (víme, co je největší společný dělitel dvou přirozených čísel?)
- **Řešení:**
 - Popišme postup tak, aby byl použitelný pro dvě libovolná přirozená čísla:
 - označme zadaná čísla x a y a menší z nich d
 - není-li d společným dělitelem x a y , pak d zmenšíme o 1, test opakujeme a skončíme, až d bude společným dělitelem x a y
- **Poznámka:**
 - Význam symbolů x , y a d použitých v algoritmu:
 - jsou to proměnné (paměťová místa), ve kterých je uložena nějaká hodnota, která se může v průběhu výpočtu měnit.

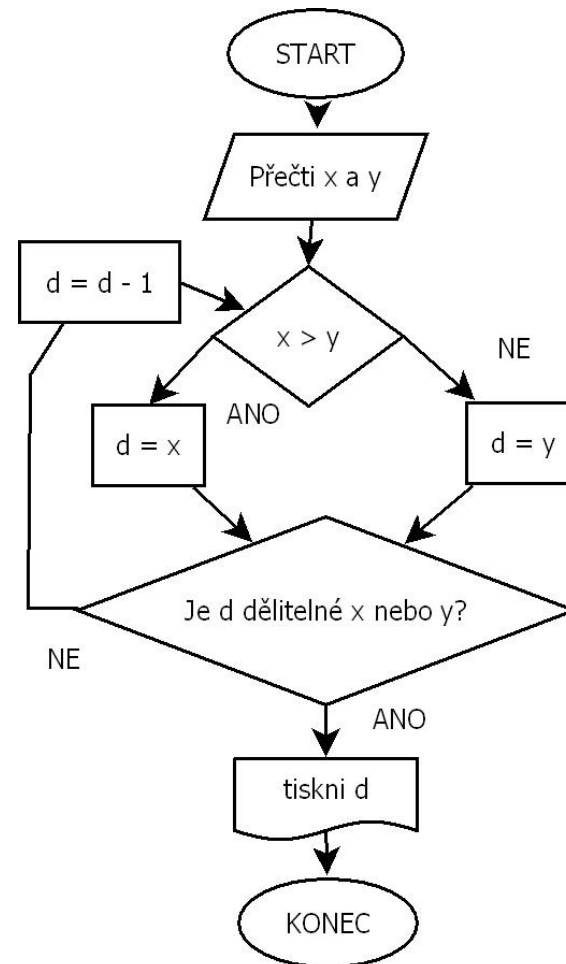
Algoritmus pro největšího spol.dělitele

Úloha: najděte největšího společného dělitele (NSD) dvou přirozených čísel

Přesnější popis:

- **Vstup:** přirozená čísla x a y
- **Výstup:** $\text{NSD}(x,y)$
- **Postup:**
 1. Je-li $x < y$, pak d má hodnotu x , jinak d má hodnotu y
 2. Pokud je d dělitelem x nebo je d dělitelem y , jdi na krok 5
 3. Zmenši d o 1
 4. Jdi na krok 2
 5. Výsledkem je hodnota d
- Sestavili jsme algoritmus pro výpočet největšího společného dělitele dvou přirozených čísel.

Zápis algoritmu pomocí vývojového diagramu



Zápis algoritmu NSD v C

```
#include "stdio.h"

int main() {
    int x, y, d = 0;
    printf("Zadej dve cisla: ");
    scanf_s("%d %d", &x, &y); /* přečteme dvě čísla */
    /* Pokud x < y, pak d má hodnotu x, jinak d má hodnotu y */
    if (x < y) d = x; else d = y;
    /* Pokud je d dělitelem x nebo je d dělitelem y, jdi na krok 5 */
    while (!(x % d) == 0 && (y % d) == 0)) d -= 1;
    /* Výsledkem je hodnota d */
    printf("NSD: %d\n",d);
    return 0;
}
```


Struktura programu v C

- Program v C realizující řešení problému je sada definic funkcí, jedna z funkcí se jmenuje **main** (nebo podobně – to stanoví konkrétní prostředí) a ta představuje hlavní program, který se spustí. Funkce **main** pak případně volá jiné funkce, či metody realizované ve třídách.
- Definice funkce **main** má např. tvar:

```
#include <stdio>
```

```
int main() {  
    int hodn = 0;  
    char volba;  
    printf("\nPříklad - tisk hodnoty do zadání nuly\n");  
    do {  
        scanf("%c",&volba);  
        printf("Hodnota = %d\n", hodn);  
    } while (volba != '0');  
    printf("Konec\n");  
    return 0;  
}
```

Různé styly řešení problémů

- Zápis programu realizuje určitou funkci či algoritmus, které představují řešení určitého problému.
- Stejný problém může mít mnoho různých řešení.
- K řešení problému můžeme dospět různými způsoby – styly.
- Dva základní styly, kterými se budeme zabývat jsou strukturovaný styl a objektově-orientovaný styl.
- Každý z nich se hodí na řešení trochu jiných problémů.
- Strukturovaný styl se hodí pro řešení procesních problémů, kdy řešení skládáme z posloupností jednotlivých kroků. Řešení je poskládáno ze strukturovaných konstrukcí nad strukturovanými daty.
- Objektově-orientovaný styl se hodí spíše pro řešení různých informačních, či návrhových systémů. Řešení je realizováno jako síť komunikujících objektů.

Stručný úvod do strukturovaného stylu

- Strukturovaný model výpočtu si představuje řešení jako sadu strukturovaných dat, nad kterými pracují strukturované funkce.
- Strukturovaný zde znamená složený z jednodušších složek systematickým způsobem.
- Na úrovni dat to znamená, že data se skládají z primitivních prvků, nebo dat strukturovaných z jiných prvků do polí, struktur nebo tříd.
- Na úrovni funkcí to znamená, že funkce se skládají z primitivních příkazů, nebo příkazů strukturovaných pomocí do sekvencí (posloupností), selekcí (podmíněných konstrukcí) nebo iterací (cyklů).
- Příklad: Představme si řešení fronty zákazníků. Především navrhne datovou reprezentaci fronty. Nad ní pak definujeme mimo jiné funkci `zarad_zakaznika`, která do fronty zařadí dalšího zákazníka. Systém, který simuluje obsluhu zákazníků vytvoří nového zákazníka (data typu `ZAKAZNIK`) a nechá ho zařadit do fronty zákazníků. Z fronty pak vybírá zákazníky, které bude obsluhovat (pomocí dalších funkcí).

Elementární algoritmické strukturované konstrukty

Posloupnost (sekvence):

- krok v algoritmu, který je považován za jednotku (nemusí být elementární operací) a je proveden pouze jednou,
- v případě více posloupností je jasné dáno pořadí jejich zpracování.

Větvení, výběr (selekce):

- podmíněný výběr průběhu algoritmu na základě vyhodnocení výběrové složky.

Cyklus (iterace):

- opakování části algoritmu (těla cyklu), které je prováděno dokud je/není splněna podmínka opakování.

Dílčí algoritmus (podprogram, funkce, metoda):

- část algoritmu, kterou lze použít opakovaně.

Výpočetní problém - příklad

- **Úloha:** najděte k danému přirozenému číslu nejbližší větší prvočíslo (víme, co je prvočíslo?)
- **Řešení:**
 - Popište postup tak, aby byl použitelný pro libovolné přirozené číslo x :
 - uložíme do buňky y zadané číslo x zvětšené o 1 (hledáme větší prvočíslo),
 - pokud není y prvočíslo, pak zvětšíme y o 1 a test opakujeme; skončíme, je-li y prvočíslo – to je pak výstupem algoritmu.
- **Poznámka:**
 - Význam symbolů x a y použitých v algoritmu:
 - jsou to proměnné (paměťová místa), ve kterých je uložena nějaká hodnota, která se může v průběhu výpočtu měnit.

Algoritmus pro nejbližší větší prvočíslo

Úloha: najděte k danému přirozenému číslu nejbližší větší prvočíslo

Přesnější popis:

- Vstup: přirozené číslo x
- Výstup: prvočíslo y takové, že $y > x$ a neexistuje prvočíslo $z < y$ a $z > x$
- Postup (pseudokód):
 1. Položme $y = x + 1$
 2. Pokud je y prvočíslo, jdi na krok 4.
 3. Pokud není, nastav $y = y + 1$ a jdi na krok 2.
 4. Výsledkem je hodnota y .
- Sestavili jsme algoritmus pro výpočet nejbližšího většího prvočísla.
- Otázka: umíme zjistit, zda je y prvočíslo?
- A pokud ano, je výše uvedený postup algoritmus?

Algoritmus pro nejbližší větší prvočíslo II.

Podúloha 1: jak ověříme, že dané přirozené číslo y je prvočíslo.

Prvočíslo je dělitelné pouze samo sebou a 1. ověření lze učinit např. tak, že vyzkoušíme všechna čísla od 2 do $y - 1$, žádné z nich nesmí být dělitelem y .

Pozn.: Lze ověřit efektivněji, na základě další znalosti o dělitelnosti (lze použít zkrácený průchod do $y/2$ či dokonce do $\sqrt{y}$).

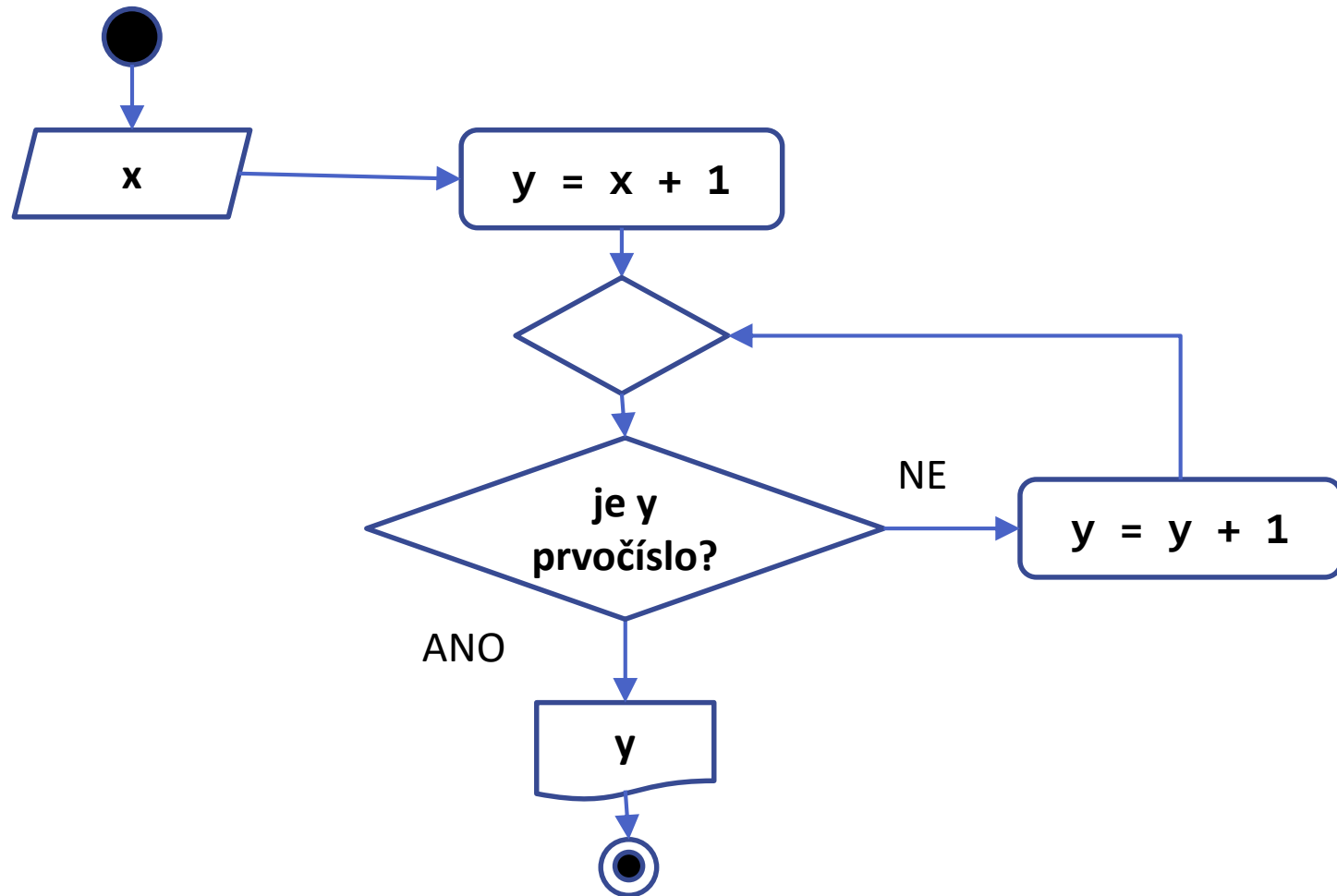
Podúloha 2: ověřte, že dříve uvedený postup pro nalezení nejbližšího prvočísla je algoritmus.

Řešení:

Pokud má být daný postup algoritmem, musí být a) deterministický, b) hromadný a c) konečný.

- a) Postup je deterministický, neboť v každém okamžiku víme, který krok se bude provádět jako následující.
- b) Postup je hromadný, neboť funguje pro libovolné přirozené číslo.
- c) Postup je konečný, neboť pro každé přirozené číslo existuje právě jedno prvočíslo, které je větší. Postupujeme-li s y směrem nahoru, musíme k tomuto prvočíslu dojít.

Zápis pomocí diagramu aktivity (UML)



Zápis algoritmu v C

```
int isPrime(int x) {  
    int res = 1;  
    for (auto i = 2; i < x; i++) if (x % i == 0) return 0;  
    return res;  
};  
  
int main() {  
    int x,y;  
    printf("Zadej prirodzene cislo: ");  
    scanf("%d", &x);  
    y = x + 1;  
    while (!isPrime(y)) y++;  
    printf("Reseni: %d\n", y);  
    system("pause");  
    return 0;  
};
```

Zápis programu v C

- Program v C je sada funkcí.
- Jedna funkce se povinně jmenuje **main** – to je hlavní program.
- Ve funkci můžeme volat jiné funkce (knihovní funkce, uživatelsky definované funkce, atd.).
- Abychom uměli funkci správně zavolat, musíme znát její profil (deklaraci, signaturu, hlavičku).
- Deklarace funkcí umísťujeme do hlavičkových souborů (soubory s příponou .h).
- Definice funkcí (vlastní programy) umísťujeme do zdrojových souborů (souborů s příponou .c, resp. .cpp).

Hlavičkové soubory

```
#ifndef __hlavicka_h
#define __hlavicka_h
int isPrime(int x)
#endif
```

nebo:

```
#pragma once
int isPrime(int x)
```

Soubor hlavicka.h

```
#include „hlavicka.h“
int isPrime(int x) {
    int res = 1;
    for (auto i = 2; i < x;)
        if (x % i) i++; else return 0;
    return res;
};
```

Soubor hlavicka.c

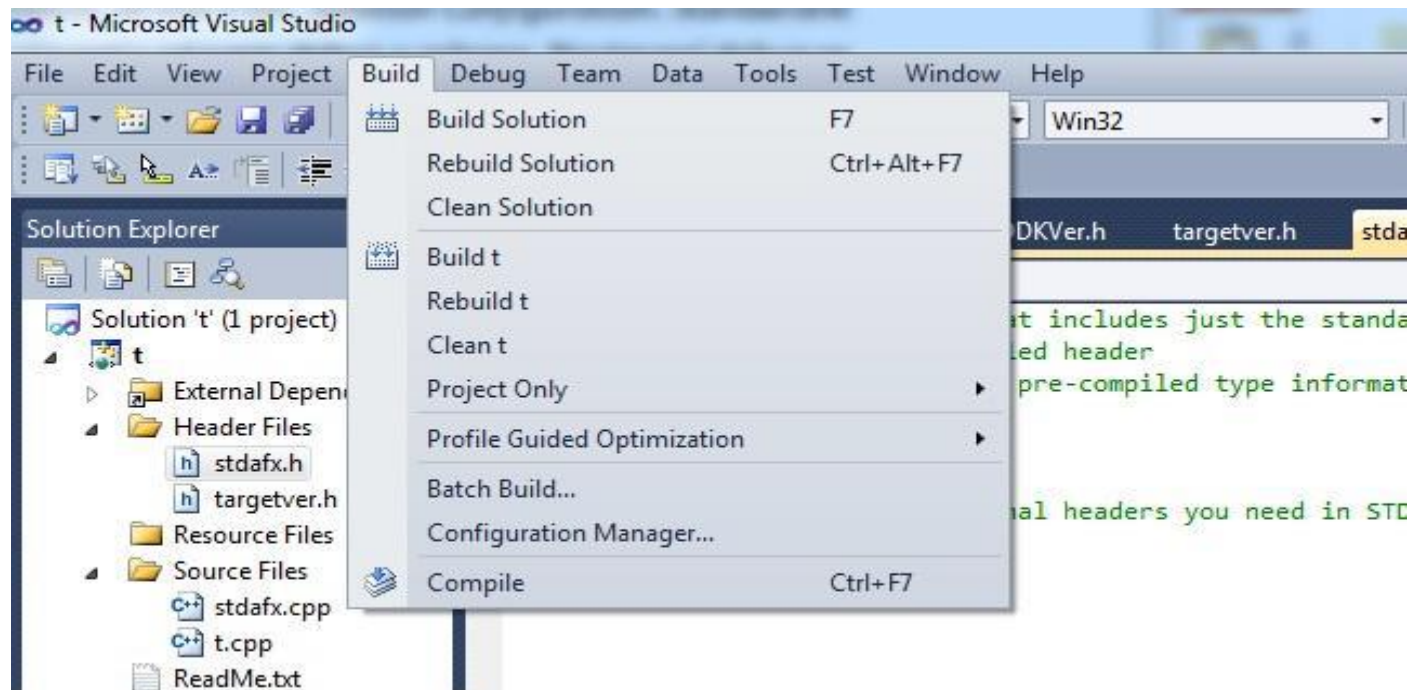
```
#include „hlavicka.h“
// funkce main
int main() {
    int x,y;
    printf("Zadej prirodzene cislo: ");
    scanf("%d", &x);
    y = x + 1;
    while (!isPrime(y)) y++;
    printf("Reseni: %d\n", y);
    system("pause");
    return 0;
};
```

Soubor main.c

Kompilace programu ve Visual studiu

Kompilace programu:

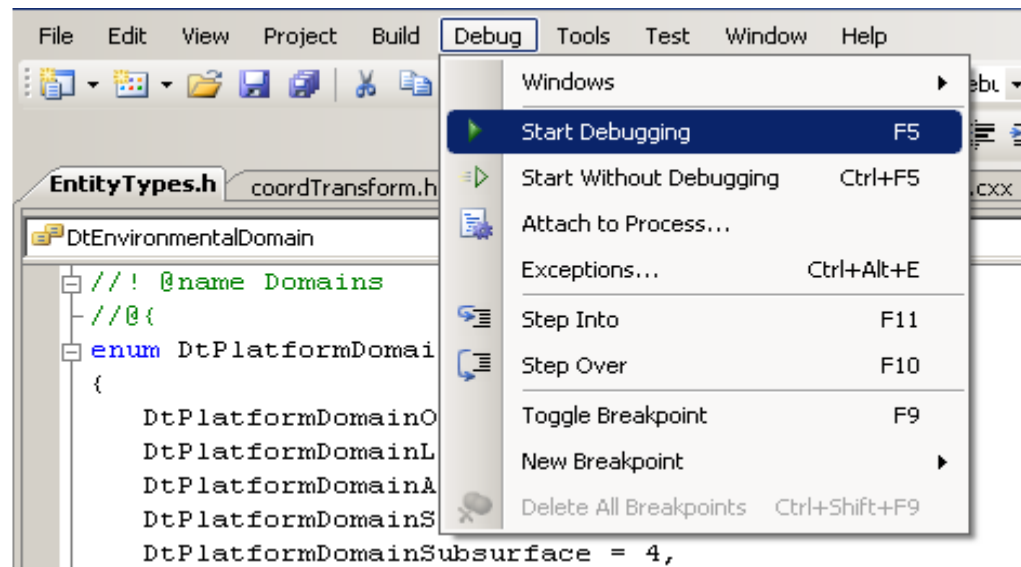
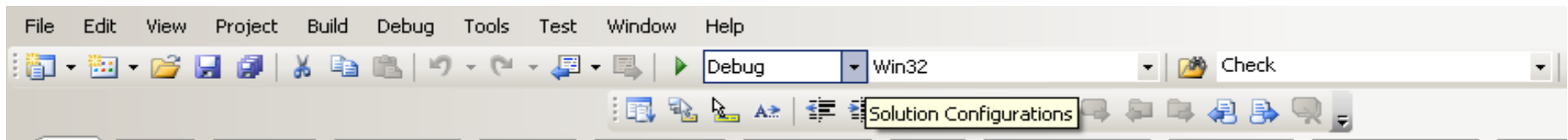
- Build – provede kompilaci a sestavení jen změněných částí
- Rebuild – všechny zdrojové kódy
- Compile – kompilace právě editovaného kódu, objektový kód



Ladění programu ve Visual studiu

Kompilace programu: Configuration Manager

- Debug – detailní ladění
- Release



The End